

PREDICTIVE MODELING FOR FY27 STORE EXPANSION

Sometime before mid-2025 I got involved with a friend on a dataset about Store Expansion Predictive Modelling Project. Our primary objective was to develop a reliable Machine Learning infrastructure (Linear Regression) to forecast expected Sales for proposed retail locations, mitigating the financial risks associated with multi-million-dollar capital investments in new regional markets. This is because if we choose a location with poor demographics or high market saturation, we risk massive financial losses.

We tasked ourselves that building a reliable Linear Regression model to forecast expected Sales for any proposed new store will, ideally, be based on its environmental, demographic, and physical attributes.

So, we mapped this project to be executed in two distinct phases. Below is our roadmap.

Phase 1:

Baseline Model Development: Our first task was to train a predictive model using historical footprints. We were provided with a dataset of 1,000 existing stores.

Execution Steps:

Target Variable: Our model had to predict total expected Sales.

Data Cleaning: Algorithms cannot process high-cardinality text easily. So, we had to drop identifiers that create noise: State, City, Store_ID, and Opening_Date.

Categorical Encoding: We had to convert the remaining categorical variables (Location_Type and Weather_Conditions) into a binary, machine-readable format using One-Hot Encoding. We dropped the first category to avoid the dummy variable trap.

Goal: We trained the model and evaluated its performance. We were aiming for an R2 score of approximately 0.75 (meaning the model should explain 75% of the variance in historical sales). Visual plots (Actual vs. Predicted, and Residual Distribution) were generated to verify the model's accuracy.

Phase 2:

Production Forecasting: Once our model was trained, we then used 10 newly proposed, empty lots in various cities as our new prediction dataset.

Execution Steps:

Data Prep: We applied the exact same preprocessing steps to this new 10-row dataset.

Column Alignment: We explicitly aligned the new data's columns to perfectly match the training data's structure before predicting.

Prediction: We ran the 10 new locations through our trained model and appended a new column called Predicted_Sales to the dataset.

Our model performed excellently on the 1,000 historical stores, and on the 10 new stores. This meant the new geographic markets behaved fundamentally the same as our old markets.

IMPORTING NECESSARY LIBRARIES

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score
```

1. LOADING THE DATA

Read the dataset into a pandas DataFrame

```
df = pd.read_csv('store_raw_data.csv')
```

df

	Store_ID	State	City	Location_Type	Population_Density	\
0	0	MD	Laurel	Rural	4421	
1	1	MI	Lansing	Urban	1003	
2	2	NY	Albany	Urban	21636	
3	3	MI	Detroit	Urban	6186	
4	4	NC	High Point	Rural	29822	
..	...	...	...	...	...	
995	995	MI	Ann Arbor	Suburban	6373	
996	996	MI	Ann Arbor	Urban	39506	
997	997	MD	Annapolis	Rural	2378	
998	998	NY	Albany	Urban	21542	
999	999	IL	Chicago	Urban	23693	

	Number_of_Competitors	Average_Income_per_capita	Weather_Conditions	\
0	8	85246	Cold	
1	6	72018	Cold	
2	2	87882	Hot	
3	9	91333	Moderate	
4	6	93144	Moderate	
..	...	...	...	
995	2	93333	Moderate	
996	5	86823	Cold	
997	1	74421	Hot	
998	9	95159	Moderate	
999	7	87378	Moderate	

	Store_Size	Parking_Space	Opening_Date	Sales
0	1994	22	6 10 2020	292504.0204
1	1006	50	7 06 2021	349046.7914
2	2112	49	13 08 2024	387145.6673
3	1162	98	20 02 2021	366571.7988
4	1495	32	17 09 2022	344911.7095
..	...	...	...	...
995	1797	60	28 04 2019	326416.6190
996	1952	34	18 04 2019	352353.3452
997	1965	80	26 09 2022	289307.2569
998	1555	29	8 05 2020	285571.6905
999	1530	25	30 07 2020	331597.4823

[1000 rows x 12 columns]

2. DATA PREPROCESSING

Drop 'State' and 'City' due to high cardinality, along with non-predictive columns

```
cols_to_drop = ['State', 'City', 'Store_ID', 'Opening_Date']
df_processed = df.drop(columns=cols_to_drop)
```

One-hot encode the remaining categorical columns

```
cat_cols = ['Location_Type', 'Weather_Conditions']
df_processed = pd.get_dummies(df_processed, columns=cat_cols,
drop_first=True, dtype=int)
```

3. DEFINING FEATURES (X) AND TARGET (y)

X contains all independent variables; y contains the dependent variable

```
X = df_processed.drop(columns=['Sales'])
```

```
y = df_processed['Sales']
```

4. TRAIN-TEST SPLIT

Splitting data into 80% training and 20% testing

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)
```

5. MODEL TRAINING

Initializing and train the Linear Regression model

```
model = LinearRegression()
model.fit(X_train, y_train)
```

```
LinearRegression()
```

6. MODEL EVALUATION

```
# Make predictions on the unseen test data
```

```
y_pred = model.predict(X_test)
```

```
# Calculate metrics
```

```
r2 = r2_score(y_test, y_pred)
```

```
rmse = np.sqrt(mean_squared_error(y_test, y_pred))
```

```
print(f"R-squared: {r2:.4f}")
```

```
print(f"Root Mean Squared Error (RMSE): {rmse:.2f}")
```

```
R-squared: 0.7598
```

```
Root Mean Squared Error (RMSE): 24727.56
```

7. VISUALIZATIONS

```
sns.set_theme(style="whitegrid")
```

```
# --- Plot 1: Actual vs Predicted Sales ---
```

```
plt.figure(figsize=(8, 6))
```

```
plt.scatter(y_test, y_pred, alpha=0.6, color='dodgerblue', edgecolor='k')
```

```
# Draw the line of perfect prediction
```

```
min_val = min(y_test.min(), y_pred.min())
```

```
max_val = max(y_test.max(), y_pred.max())
```

```
plt.plot([min_val, max_val], [min_val, max_val], 'r--', lw=2, label='Perfect Prediction')
```

```
plt.xlabel('Actual Sales')
```

```
plt.ylabel('Predicted Sales')
```

```
plt.title('Linear Regression: Actual vs Predicted Sales')
```

```
plt.legend()
```

```
plt.tight_layout()
```

```
plt.show()
```

```
# --- Plot 2: Distribution of Residuals ---
```

```
residuals = y_test - y_pred
```

```
plt.figure(figsize=(8, 6))
```

```
sns.histplot(residuals, kde=True, color='purple', bins=30)
```

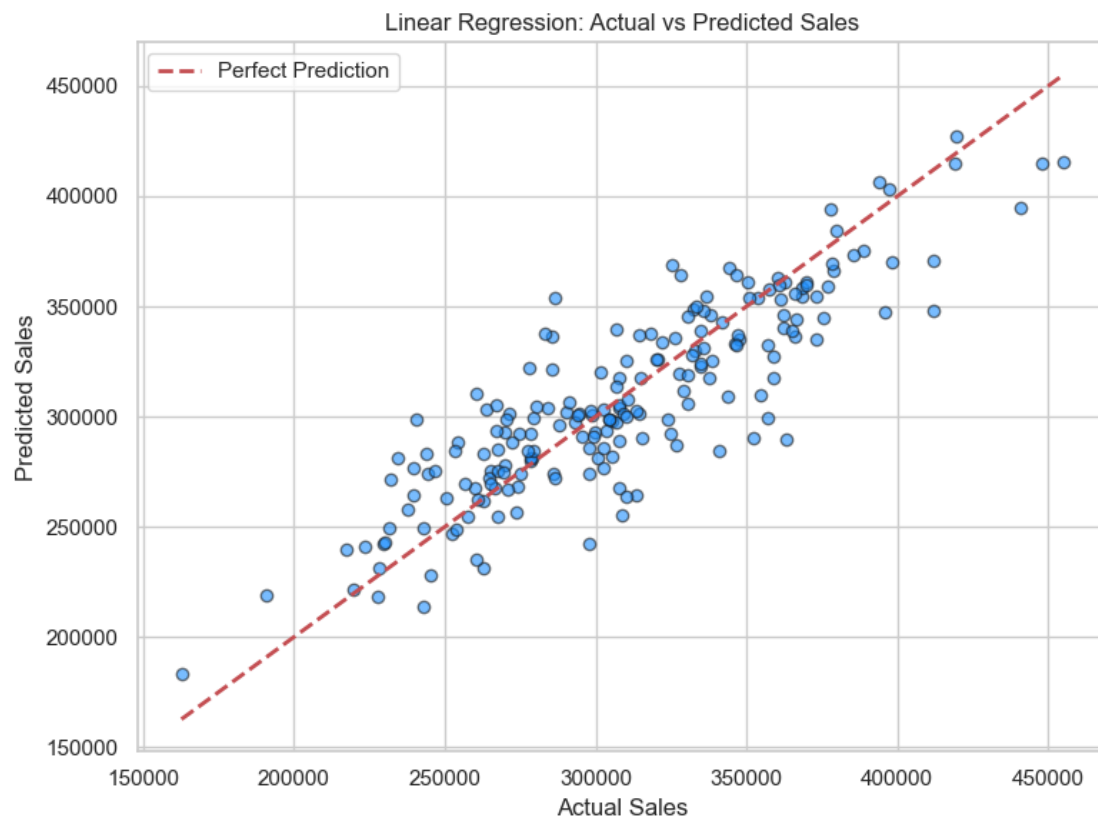
```
plt.axvline(x=0, color='r', linestyle='--', lw=2)
```

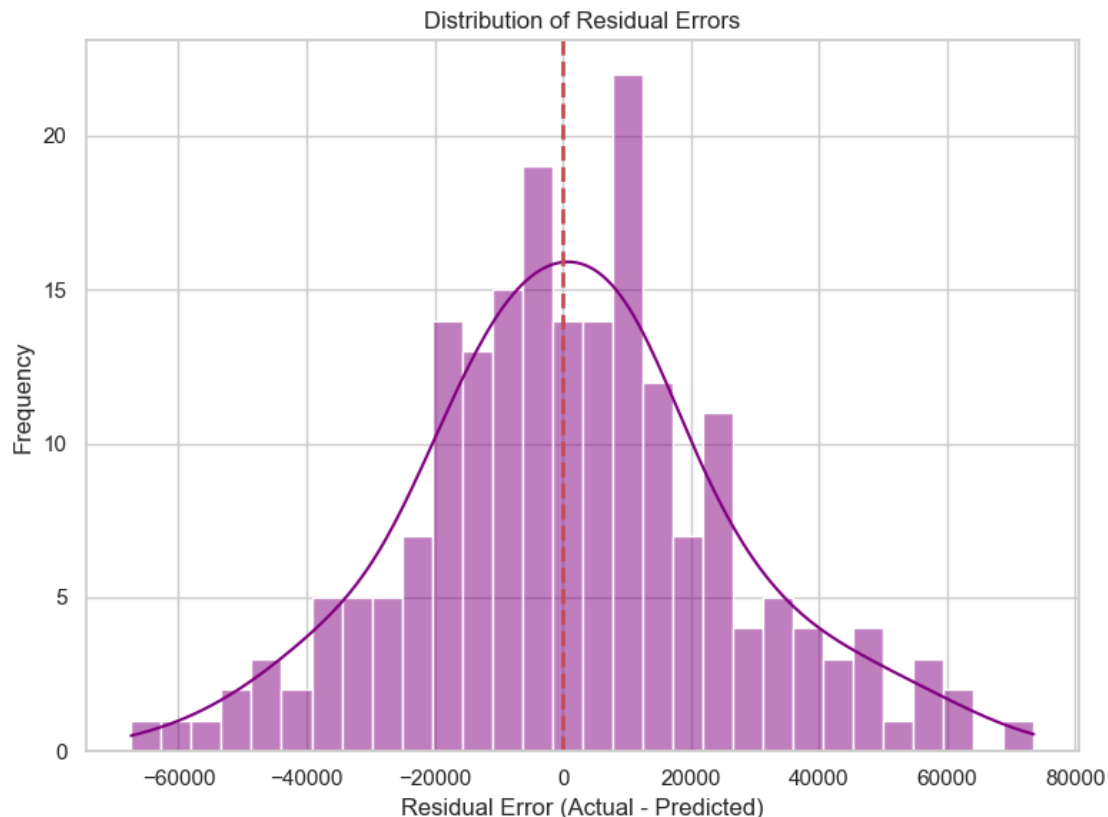
```
plt.xlabel('Residual Error (Actual - Predicted)')
```

```
plt.ylabel('Frequency')
```

```
plt.title('Distribution of Residual Errors')
```

```
plt.tight_layout()  
plt.show()
```





PART 2|

1. RE-TRAINING THE ORIGINAL MODEL

```
import pandas as pd
import numpy as np
from sklearn.linear_model import LinearRegression

# Load the ORIGINAL training data
df_train_proc = pd.read_csv(store_raw_data.csv')

# Drop the text/ID columns we don't need
cols_to_drop = ['State', 'City', 'Store_ID', 'Opening_Date']
df_train_proc = df_train_proc.drop(columns=cols_to_drop)

# Convert Categories into 1s and 0s
df_train_proc = pd.get_dummies(df_train_proc, columns=['Location_Type',
'Weather_Conditions'], drop_first=True, dtype=int)

# Separate the Features (X) from the Target (y)
X_train = df_train_proc.drop(columns=['Sales'])
y_train = df_train_proc['Sales']
```

```
# Train the Model!
model = LinearRegression()
model.fit(X_train, y_train)
```

```
LinearRegression()
```

2: LOADING NEW UNSEEN DATA

```
# Load the NEW dataset
df_new = pd.read_csv(store_raw_data2.csv')
```

```
# Make a copy to hold our final output
df_out = df_new.copy()
```

```
df_new
```

	Store_ID	State	City	Location_Type	Population_Density	\
0	1001	MI	Lansing	Urban	36164	
1	1002	FL	Orlando	Suburban	4746	
2	1003	IL	Chicago	Urban	35239	
3	1004	WA	Seattle	Rural	4327	
4	1005	OH	Columbus	Rural	1505	
5	1006	CA	Los Angeles	Rural	3130	
6	1007	OH	Columbus	Suburban	9177	
7	1008	TX	Austin	Suburban	8965	
8	1009	MD	Laurel	Rural	2746	
9	1010	TX	Austin	Suburban	7183	

	Number_of_Competitors	Average_Income_per_capita	Weather_Conditions	\
0	11	83466	Cold	
1	5	76564	Cold	
2	6	97081	Moderate	
3	0	90979	Cold	
4	1	30000	Cold	
5	1	53527	Hot	
6	0	110928	Cold	
7	5	80418	Moderate	
8	0	74827	Moderate	
9	3	93960	Cold	

	Store_Size	Parking_Space	Opening_Date	Sales
0	800	15	15-03-2023	245154.1205
1	1077	30	09-12-2023	221154.5512
2	1352	25	15-01-2023	380028.9811
3	1574	46	19-02-2023	338627.4201
4	2057	61	08-04-2023	223251.3488
5	1750	60	19-08-2023	230194.8872
6	1480	47	03-06-2023	354797.1009
7	1402	37	17-03-2024	273708.2045

8	1088	34	24-10-2023	277781.9904
9	2254	67	18-01-2023	380392.3397

3: PREPROCESSING

Drop the same unnecessary columns

```
df_new_proc = df_new.drop(columns=cols_to_drop)
```

*# If this new dataset happens to have a 'Sales' column (Like an answer key),
we must hide it from the model so it doesn't cheat!*

```
if 'Sales' in df_new_proc.columns:
    df_new_proc = df_new_proc.drop(columns=['Sales'])
```

Convert the text categories into 1s and 0s again

```
df_new_proc = pd.get_dummies(df_new_proc, columns=['Location_Type',
'Weather_Conditions'], drop_first=True, dtype=int)
```

4 ONE-HOT ENCODING / COLUMN ALIGNMENT

Align the columns of the new data to perfectly match the training data

```
X_new = df_new_proc.reindex(columns=X_train.columns, fill_value=0)
```

5 MAKING PREDICTIONS

Predicting the Sales

```
predictions = model.predict(X_new)
```

Adding the predictions to output file (rounded to 4 decimal places)

```
df_out['Predicted_Sales'] = np.round(predictions, 4)
```

Saving the completed file as a CSV

```
output_file = 'predictions_new_data.csv'
df_out.to_csv(output_file, index=False)
```

```
print("Predictions successfully generated and saved!")
```

```
print(df_out)
```

Predictions successfully generated and saved!

	Store_ID	State	City	Location_Type	Population_Density \
0	1001	MI	Lansing	Urban	36164
1	1002	FL	Orlando	Suburban	4746
2	1003	IL	Chicago	Urban	35239
3	1004	WA	Seattle	Rural	4327
4	1005	OH	CoLumbus	Rural	1505
5	1006	CA	Los Angeles	Rural	3130
6	1007	OH	Columbus	Suburban	9177
7	1008	TX	Austin	Suburban	8965

8	1009	MD	Laurel	Rural	2746
9	1010	TX	Austin	Suburban	7183

	Number_of_Competitors	Average_Income_per_capita	Weather_Conditions	\
0	11	83466	Cold	
1	5	76564	Cold	
2	6	97081	Moderate	
3	0	90979	Cold	
4	1	30000	Cold	
5	1	53527	Hot	
6	0	110928	Cold	
7	5	80418	Moderate	
8	0	74827	Moderate	
9	3	93960	Cold	

	Store_Size	Parking_Space	Opening_Date	Sales	Predicted_Sales
0	800	15	15-03-2023	245154.1205	279759.3409
1	1077	30	09-12-2023	221154.5512	233287.8081
2	1352	25	15-01-2023	380028.9811	372062.3468
3	1574	46	19-02-2023	338627.4201	293874.9432
4	2057	61	08-04-2023	223251.3488	212640.9763
5	1750	60	19-08-2023	230194.8872	245853.7285
6	1480	47	03-06-2023	354797.1009	344263.4157
7	1402	37	17-03-2024	273708.2045	282156.3402
8	1088	34	24-10-2023	277781.9904	261936.6256
9	2254	67	18-01-2023	380392.3397	334146.7640

Results

Looking at the predictions, our model made on the new data:

- It predicted the proposed store in Chicago (Store 1003) would make a massive \$372,062.
- It predicted the proposed store in Columbus (Store 1005) would only make \$212,640.

Because we built this model, we now know that the construction in Chicago will be approved and the location in Columbus will be rejected. We have just taken care of a bad investment situation.

This was what this dataset and the project were all about: using the patterns of the past (1,000 rows) to predict and optimize the future (10 rows).